

بازنگری نهایی پروپوزال

AI Fusion Scheduler

Linux sched_ext روی Deadline-aware Scheduling + Admission Control

وضعیت: پیاده‌سازی و ارزیابی نهایی تکمیل شده

شاخه مستندسازی: paper-finalization

Experimental freeze: experimental-freeze-v1

Freeze commit: edcd8eb347f998fc68c705ca770eaa645c8e78be

Results documentation commit: b39c84d044bf4d942bcb0e64f0b6a6ae4b7a4972

1. چکیده

این پروژه یک زمان‌بند آزمایشی برای بارهای کاری ناهمگن طراحی و پیاده‌سازی می‌کند که در آن taskهای critical، interactive و batch می‌توانند deadline، اولویت، رفتار bursty و فشار CPU متفاوت داشته باشند. هدف اصلی این است که در شرایط saturation و overload، به‌جای تلاش برای تکمیل همه کارها، کیفیت سرویس taskهای deadline‌دار و پذیرفته‌شده حفظ شود و overload به‌صورت کنترل‌شده مدیریت شود.

راهکار پیشنهادی، **AI Fusion Scheduler (AFS)**، روی Linux sched_ext پیاده‌سازی شده و سه جزء اصلی را ترکیب می‌کند: یک policy مبتنی بر laxity/deadline/priority/aging، یک admission controller آگاه از deadline و workload-scoped CPU PSI، و یک harness بازتولیدپذیر برای مقایسه با stock، EEVDF، RustLand، AFS بدون admission و SCHED_DEADLINE در workloadهای کنترل‌شده.

ارزیابی نهایی شامل 120 اجرای primary و 70 اجرای measured secondary به‌علاوه 10 اجرای manifest-seed است. تمام 80 validation استاندارد secondary با وضعیت OK پایان یافته‌اند. نتایج نشان می‌دهند AFS در بار سبک رفتار نزدیک به baseline دارد؛ در saturation تقریباً goodput مشابه RustLand ارائه می‌کند ولی admission control، tail latency، accepted miss و pressure را با هزینه rejection کاهش می‌دهد؛ و در overload با admission control بخش بزرگی از بار را رد می‌کند تا deadline goodput را و tail latency task پذیرفته‌شده بهتر حفظ شود. این trade-off عمداً در نتایج برجسته می‌شود و به‌عنوان «بهبود بدون هزینه» گزارش نمی‌شود.

2. بیان مسئله

بارهای محاسباتی جدید - از pipelineهای چندمرحله‌ای داده و AI تا سرویس‌های interactive و taskهای background - الزاماً یک هدف زمان‌بندی واحد ندارند. یک سیستم ممکن است هم‌زمان شامل taskهایی با deadline نرم، taskهای latency-sensitive، کارهای batch و workflowهای چندمرحله‌ای باشد. در چنین شرایطی، معیار throughput به‌تنهایی کافی نیست؛ scheduler باید بداند کدام کار در معرض از دست دادن deadline است، کدام کار می‌تواند عقب بیفتد، و چه زمانی ظرفیت سیستم دیگر برای پذیرش همه بارها کافی نیست.

در overload، اگر scheduler همه taskها را بدون admission بپذیرد، queueing delay افزایش می‌یابد و حتی taskهایی که در صورت کنترل بار می‌توانستند deadline خود را رعایت کنند نیز دیر تکمیل می‌شوند. بنابراین مسئله اصلی پروژه این است:

چگونه می‌توان با ترکیب scheduling policy و deadline goodput، admission control را در saturation و overload بهبود داد، بدون اینکه هزینه rejection و completion پنهان شود؟

3. اهداف پروژه

3.1 هدف اصلی

طراحی، پیاده‌سازی و ارزیابی یک scheduler مبتنی بر sched_ext که برای workloadهای ناهمگن بتواند با استفاده از metadata کاربردی، aging، deadline/laxity و admission control، رفتار overload را کنترل کند و کیفیت taskهای deadline دار را حفظ نماید.

3.2 اهداف فرعی

- جداسازی اثر scheduling policy از admission control.
- استفاده از workload-scoped CPU PSI به جای PSI سراسری سیستم برای جلوگیری از آلودگی سیگنال توسط control plane کاربر.
- اعمال واقعی CPU isolation و affinity برای workload و control plane.
- پشتیبانی از workloadهای light، saturated، overload-bursty و periodic/sporadic.
- مقایسه با stock RustLand، EEVDF و Linux SCHED_DEADLINE در سناریوی مناسب.
- اجرای ablation برای aging و application-deadline awareness.
- اجرای یک مطالعه موردی Massive Data Fusion برای نشان دادن مسیر end-to-end.
- ارائه harness بازتولیدپذیر با shared manifests، validation، provenance و aggregation آماری ثابت.

4. پرسش‌های پژوهش

1. آیا AFS در saturation و overload می‌تواند deadline goodput را با trade-off شفاف admission حفظ یا بهبود دهد؟
2. چه بخشی از رفتار مشاهده شده ناشی از scheduling policy است و چه بخشی ناشی از admission control؟
3. حذف aging چه اثری بر overload و accepted deadline misses دارد؟
4. حذف application-deadline awareness چه اثری بر goodput و miss rate دارد؟
5. AFS در workload periodic/sporadic در مقایسه با EEVDF و SCHED_DEADLINE چه رفتاری دارد؟
6. آیا معماری پیشنهادی در یک pipeline چندمرحله‌ای Fusion به صورت end-to-end قابل اجرا است؟

5. نوآوری و مشارکتهای فنی

5.1 Scheduling policy مبتنی بر metadata و laxity

AFS برای تصمیم‌گیری از deadline/laxity، اولویت، aging و کلاس task استفاده می‌کند. برای taskهای deadline دار، slack/laxity نقش مستقیم در urgency دارد و برای taskهای base laxity، batch جداگانه در نظر گرفته شده است. aging نیز مانع کنار گذاشته شدن طولانی مدت taskهای کم‌فوریت می‌شود.

5.2 Admission control آگاه از deadline و pressure

Admission controller با استفاده از workload-scoped CPU PSI و تخمین runtime تصمیم می‌گیرد task جدید پذیرفته، reject یا defer شود. در نسخه نهایی، admission تنها یک آستانه فشار ساده نیست؛ deadline safety factor و feasibility نیز در تصمیم دخیل‌اند.

5.3 Workload-scoped PSI

سیگنال `/proc/pressure/cpu` برای این پروژه مناسب نبود، زیرا `userspace sched_ext control plane` می‌توانست حتی در غیاب workload واقعی فشار قابل‌توجه ایجاد کند. در نسخه نهایی، workerها در cgroup اختصاصی workload قرار می‌گیرند و admission از `cpu.pressure` همان cgroup استفاده می‌کند.

5.4 CPU partitioning واقعی

چهار CPU برای workload (2,3,4,5) و دو CPU برای control plane (0,1) در VM نهایی اختصاص داده شدند. این جداسازی صرفاً یک مقدار config نیست؛ affinity و cpuset به‌صورت واقعی enforce شده‌اند.

5.5 Harness بازتولیدپذیر

ارزیابی نهایی از shared manifests، repetition-matched runs، validation per run، provenance، pinned scx revision، freeze commit، aggregation آماری ثابت استفاده می‌کند. Pilot و diagnostic از ادعاهای نهایی حذف شده‌اند.

6. معماری سیستم

مسیر اجرایی AFS شامل اجزای زیر است:

1. **Workload/Fusion generator**: ایجاد taskها براساس config و manifest ثابت.
2. **Metadata manager**: نگهداری workflow و deadline، class، priority، runtime estimate و metadata.
3. **Policy core**: محاسبه urgency/score و ترتیب dispatch.
4. **Admission controller**: استفاده از deadline feasibility و workload PSI برای accept/defer/reject.
5. **Userspace sched_ext scheduler**: اعمال policy از طریق RustLand/scx infrastructure.
6. **Analysis pipeline**: استخراج plots، validation، Student-t CI، run-level metrics و curated snapshots.

7. روش ارزیابی

7.1 محیط

- Apple Silicon روی UTM در Debian 13 ARM64
- Kernel: 7.1.3+deb13-arm64

- شش CPU مجازی: workload روی 2-5 و control plane روی 0-1

- sched_ext Linux و cgroup v2

- 7a58a3a1a857fa748ea5a3bda5ad312afbbeca5b: scx pinned revision

7.2 روش‌های مقایسه

- EEVDF

- stock RustLand

- AFS without admission

- Full AFS

- workload periodic/sporadic در SCHED_DEADLINE

7.3 Workloadها

- **Light**: ناحیه بدون ازدحام شدید و مناسب برای بررسی no-harm.

- **Saturated**: رقابت پایدار و فشار CPU بالا.

- **Overload-bursty**: بار بسیار بیشتر از ظرفیت همراه با burstهای critical/interactive.

- **Periodic/sporadic**: سناریوی مقایسه با SCHED_DEADLINE.

- **Massive Data Fusion**: مطالعه موردی pipeline چندمرحله‌ای.

7.4 قرارداد آماری

Primary شامل 3 repetition $\times$ 4 method $\times$ workload = **run 120** است. هر repetition واحد آماری مستقل محسوب می‌شود. برای هر metric، میانگین و نیم‌عرض فاصله اطمینان دوطرفه 95% Student-t با $n=10$ ، $t=2.262$ و $df=9$ گزارش می‌شود.

Rejected deadline tasks در مخرج **offered deadline goodput** باقی می‌مانند. بنابراین goodput همیشه همراه با rejection rate، accepted miss rate و completion rate تفسیر می‌شود.

8. نتایج نهایی

Primary matrix 8.1

Workload	Method	Deadline goodput	Accepted miss	Rejection	Completion	P95 response	Workload PSI avg10
Light	EEVDF	100.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	100.00 $\pm$ 0.00%	2447.0 $\pm$ 46.6 ms	0.36 $\pm$ 0.26
Light	RustLand	100.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	100.00 $\pm$ 0.00%	2470.2 $\pm$ 52.7 ms	1.65 $\pm$ 1.40
Light	AFS no admission	100.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	0.00 $\pm$ 0.00%	100.00 $\pm$ 0.00%	2473.5 $\pm$ 55.4 ms	1.41 $\pm$ 0.41
Light	Full AFS	99.82 $\pm$ 0.41%	0.18 $\pm$ 0.41%	0.00 $\pm$ 0.00%	100.00 $\pm$ 0.00%	2469.2 $\pm$ 47.5 ms	2.04 $\pm$ 1.17

Workload	Method	Deadline goodput	Accepted miss	Rejection	Completion	P95 response	Workload PSI avg10
Saturated	EEVDF	38.87 ± 6.62%	61.13 ± 6.62%	0.00 ± 0.00%	100.00 ± 0.00%	20958.4 ± 783.1 ms	88.20 ± 5.81
Saturated	RustLand	83.57 ± 13.06%	16.43 ± 13.06%	0.00 ± 0.00%	100.00 ± 0.00%	25184.2 ± 1062.9 ms	89.81 ± 5.78
Saturated	AFS no admission	53.48 ± 8.14%	46.52 ± 8.14%	0.00 ± 0.00%	100.00 ± 0.00%	21805.7 ± 1069.8 ms	87.55 ± 5.54
Saturated	Full AFS	82.17 ± 3.08%	7.01 ± 0.94%	18.05 ± 2.34%	81.95 ± 2.34%	6186.9 ± 578.8 ms	35.12 ± 2.95
Overload	EEVDF	0.00 ± 0.00%	100.00 ± 0.00%	0.00 ± 0.00%	100.00 ± 0.00%	130276.9 ± 638.8 ms	98.68 ± 0.16
Overload	RustLand	0.00 ± 0.00%	100.00 ± 0.00%	0.00 ± 0.00%	100.00 ± 0.00%	143215.0 ± 1171.4 ms	99.17 ± 0.09
Overload	AFS no admission	1.89 ± 0.31%	98.11 ± 0.31%	0.00 ± 0.00%	100.00 ± 0.00%	130496.8 ± 623.0 ms	98.56 ± 0.11
Overload	Full AFS	13.32 ± 1.58%	37.01 ± 6.36%	79.38 ± 0.45%	20.62 ± 0.45%	7025.8 ± 662.3 ms	87.66 ± 1.89

8.2 تفسیر Primary

- **Light**: هر چهار روش تقریباً در سقف goodput هستند؛ این workload شاهد برتری نیست و به عنوان tie/no-harm تفسیر می شود.
- **Saturated**: Full AFS از نظر goodput تقریباً هم سطح RustLand است. فاصله اطمینان RustLand گسترده و هم پوشان است، بنابراین ادعای برتری آماری goodput نسبت به RustLand مطرح نمی شود. مزیت اصلی AFS در P95، accepted miss و PSI کمتر است، با هزینه حدود 18% rejection.
- **Overload**: admission رفتار سیستم را از «تکمیل همه کارها با deadline failure تقریباً کامل» به «رد بخش بزرگی از بار و حفظ کیفیت subset پذیرفته شده» تغییر می دهد. rejection حدود 79% جزء اصلی نتیجه است و نباید پنهان شود.

8.3 Ablation ها

Ablation	Saturated goodput	Overload goodput	Overload accepted miss	نکته
Full AFS	82.17%	13.32%	37.01%	مرجع
No aging	83.04%	10.56%	51.24%	به کاهش افت overload در aging کمک می کند deadline-quality
No application deadline	58.70%	0.40%	99.02%	است policy بخش مؤثر deadline/laxity

Periodic/sporadic 8.4

Method	Goodput	Accepted miss	Rejection	Completion	P95
EEVDF	99.33%	0.67%	0.00%	100.00%	111.0 ms
Full AFS	99.87%	0.13%	0.00%	100.00%	104.2 ms
SCHED_DEADLINE	96.53%	3.47%	0.00%	100.00%	147.8 ms

این نتیجه فقط برای workload periodic/sporadic frozen پروژۀ معتبر است و به عنوان برتری عمومی نسبت به همه workload های real-time لینوکس تعمیم داده نمی شود.

Fusion case study 8.5

اجرای نهایی Fusion شامل **task 120** بود: **completed 45**, **rejected 75**, **failed 0** و **deadline 19 miss**. این اجرا یک case study تک سناریویی است، نه یک گروه آماری ده تکراری؛ بنابراین برای اثبات امکان اجرای end-to-end architecture استفاده می شود و نه برای تعمیم آماری.

9. تهدیدهای اعتبار و محدودیت ها

- محیط ارزیابی یک VM ARM64 شش-CPU است؛ نتایج مستقیماً به همه معماری ها و ماشین های چند سوخته قابل تعمیم نیست.
- workload ها synthetic و config-driven هستند و نماینده کامل همه بارهای AI/production نیستند.
- admission در overload به rejection بالا متکی است؛ این هزینه باید در هر ادعای performance ذکر شود.
- overhead داخلی AFS برای ادعای مستقیم cross-method overhead کافی نیست، زیرا instrumentation روش ها یکسان نیست.
- نتیجه SCHED_DEADLINE تنها به periodic/sporadic workload frozen محدود است.
- Fusion فقط یک case study است و CI آماری ندارد.

10. خروجی های نهایی پروژه

- پیاده سازی scheduler و admission controller روی sched_ext .
- harness آزمایش، validation، provenance و shared manifests.
- primary snapshot نهایی در docs/final-results/ .
- secondary snapshot و 32 نمودار publication در docs/secondary-results/ .
- Results section انگلیسی و فارسی.
- این بازنگری نهایی پروپوزال.
- گزارش نهایی پروژه و PowerPoint نتایج به عنوان گام های بعدی مستندسازی.

11. جمع‌بندی

AFS نشان می‌دهد که در workloadهای ناهمگن، «پذیرش همه کارها» الزاماً بهترین سیاست نیست. در بار سبک، scheduler پیشنهادی رفتار baseline-like دارد. در saturation، با goodput مشابه RustLand می‌تواند miss پذیرفته‌شده، tail latency و workload pressure را با trade-off rejection کاهش دهد. در overload، admission بخش بزرگی از بار را رد می‌کند تا subset پذیرفته‌شده کیفیت deadline بهتری داشته باشد. Ablationها نیز نشان می‌دهند application-deadline awareness بخش مهمی از policy است و aging به‌خصوص در بار بسیار سنگین نقش حفاظتی دارد.

بنابراین contribution اصلی پروژه نه یک ادعای «scheduler سریع‌تر در همه شرایط»، بلکه یک معماری قابل‌آزمایش برای **deadline-aware scheduling + overload admission + reproducible evaluation** روی sched_ext است.

12. منشأ و قابلیت بازتولید

- docs/final-results/ :Primary results •
- docs/secondary-results/ :Secondary results and plots •
- docs/FINAL_RESULTS_SECTION.md :Final results section •
- report/FINAL_RESULTS_SECTION_FA.md :Persian final-results section •
- experimental-freeze-v1 :Experimental freeze tag •
- edcd8eb347f998fc68c705ca770eaa645c8e78be :Freeze commit •
- b39c84d044bf4d942bcb0e64f0b6a6ae4b7a4972 :Final results documentation commit •